

</> WEEK 2 • DAY 2

Full-Stack Application Development



Instructor

Dilovan Matini



Date

Monday, March 2, 2026

CSS Styling + Basic JavaScript Intro



DAY 1 RECAP

Review of Day 1



HTML Basics

- ✓ **Structure & Tags:** Understanding HTML document structure and common tags
- ✓ **Elements:** Headings, paragraphs, links, images, and lists
- ✓ **Attributes:** Adding extra information to HTML elements
- ✓ **Semantic HTML:** Using meaningful tags for better structure



CSS Purpose

- ✓ **Styling:** CSS controls the visual presentation of HTML
- ✓ **Separation:** Keep content (HTML) separate from design (CSS)
- ✓ **Consistency:** Apply styles across multiple pages
- ✓ **Flexibility:** Easy to update and maintain



Key Takeaway: HTML provides the structure, CSS provides the style, and together they create the foundation of every web page.



What Is CSS?

CSS (Cascading Style Sheets) is a stylesheet language used to describe the presentation of a document written in HTML. It controls how HTML elements are displayed on screen.

Think of HTML as the skeleton and CSS as the skin, hair, and clothing that make it visually appealing.



Colors

Text colors, background colors, gradients, and opacity



Fonts

Font families, sizes, weights, styles, and line heights



Spacing

Margins, padding, and element positioning



Layout

Flexbox, Grid, and responsive design

How CSS Works

1

HTML Structure

Create the content and structure

2

CSS Rules

Define styles for HTML elements

3

Browser Rendering

Browser applies styles to elements

4

Visual Output

Styled web page displayed to user

Benefit: Change the entire look of a website by modifying just the CSS file!



PRACTICAL EXAMPLES

CSS Example

CSS Syntax Structure

```
selector {  
  property: value;  
}
```

Selector

Target element

Property

What to change

Value

How to change

Example 1: Change Heading Color

```
h1 {  
  color: blue;  
}
```

☒ All elements will appear in **blue**

Example 2: Change Paragraph Size

```
p {  
  font-size: 18px;  
}
```

All elements will have:

- ✓ Font size of **18 pixels**
- ✓ More readable than default (16px)

Common CSS Properties

color

Text color

font-size

Text size

background-color

Background

margin

Outer spacing

padding

Inner spacing

border

Element border



HANDS-ON CODING

Practice CSS Exercise

Exercise Tasks

1 Change Text Color

Modify the color of headings and paragraphs

```
h1 { color: your-color; }  
p { color: your-color; }
```

2 Change Font Sizes

Experiment with different size values

```
h1 { font-size: 32px; }  
p { font-size: 16px; }
```



Live Coding: Use Cursor editor to practice in real-time

Color Values to Try



red



blue



green



#D69E2E



purple



#58A6A6

Tip: You can use:

- Named colors: red , blue
- Hex codes: #FF5733
- RGB values: rgb(255, 87, 51)

Font Size Units

px

Pixels (fixed)

em

Relative to parent

rem

Relative to root



What Is JavaScript?

JavaScript (JS) is a programming language that enables interactive web pages. It's an essential part of web applications, allowing you to implement complex features and dynamic behavior.

While HTML provides structure and CSS provides style, JavaScript provides **behavior and interactivity**.

What JavaScript Can Do



Respond to User Actions

Handle clicks, keyboard input, form submissions



Modify Content

Update text, images, and HTML dynamically



Perform Calculations

Math operations, data processing, logic



Communicate with Servers

Fetch data, send information, API calls

JavaScript Runs in the Browser



Client-Side Execution

Code runs directly in the user's browser without needing a server



Immediate Feedback

Instant response to user interactions, no page reload needed



Works with HTML & CSS

Seamlessly integrates with existing web technologies

The Web Trinity



HTML
Structure



CSS
Style



JS
Behavior



FIRST INTERACTION

Simple JavaScript Example

The alert() Function

The **alert()** function displays a popup dialog box with a message. It's one of the simplest ways to show output in JavaScript.

```
alert("Hello World!");
```

What happens:

1. Browser executes the JavaScript code
2. A popup dialog appears with "Hello World!"
3. User must click "OK" to close the dialog
4. Page execution pauses until dialog closes



Note: While useful for learning, alert() is rarely used in production due to its intrusive nature.

How to Add JavaScript

Method 1: Internal Script

Place in HTML file, usually before closing

Method 2: External File (Recommended)

```
// In HTML:  
src="script.js">  
// In script.js:  
alert("Hello!");
```

Separates logic from structure, easier to maintain

Try These Variations



STORING DATA

Variables in JavaScript

What Are Variables?

Variables are **containers for storing data values**. Think of them as labeled boxes where you can keep information to use later in your code.



Variable `x` stores the value `10`

Declaring Variables

Use **let** or **const** to create variables:

```
// Using let (value can change)
let name = "John";
name = "Jane"; // ✓ Allowed
```

```
// Using const (value cannot change)
const PI = 3.14159;
// PI = 3.14; // ✗ Error!
```

let vs const

let Mutable (Changeable)

- ✓ Use when value will change
- ✓ Good for counters, user input
- ✓ Block-scoped

const Immutable (Constant)

- ✓ Use when value stays the same
- ✓ Good for constants, configuration
- ✓ Prevents accidental changes

Best Practice

Default to const, use **let** only when you know the value will change. This prevents bugs!



Get User Input

The prompt() Function

The **prompt()** function displays a dialog box that asks the user for input. It returns the text entered by the user as a string.

```
let name = prompt("What is your name?");
```

What happens:

1. A dialog box appears with the message
2. User types their response
3. User clicks "OK" or "Cancel"
4. The input is stored in the variable `name`

Display the Result

Use **alert()** to show the user's input:

```
let name = prompt("What is your name?");  
alert("Hello, " + name + "!");
```

Complete Example

```
// Ask for user's name  
let userName = prompt("What is your name?");  
// Ask for user's age  
let userAge = prompt("How old are you?");  
// Display personalized message  
alert("Hello " + userName + "! You are " + userAge + " years old.");
```

Result Flow:

Prompt 1 → Prompt 2 → Alert Result

Important Notes

- ⚠ `prompt()` always returns a **string**
- ⚠ If user clicks "Cancel", it returns null
- ⚠ For numbers, use `parseInt()` or `parseFloat()`



TROUBLESHOOTING

Debugging Basics

Browser Developer Tools

Every modern browser includes **Developer Tools** (DevTools) that help you debug JavaScript, inspect HTML/CSS, and analyze performance.

How to Open DevTools:

F12 Press F12 key

Ctrl+Shift+J Windows/Linux

Cmd+Option+J Mac

 Right-click → Inspect

The Console Tab

The Console is your best friend for debugging:

 View error messages and warnings

Common JavaScript Errors

✗ SyntaxError

```
alert("Hello" ← Missing closing parenthesis
```

Typos, missing brackets, or incorrect syntax

✗ ReferenceError

```
alert(myVariable); ← Variable not defined
```

Using a variable that doesn't exist

✗ TypeError

```
let x = 5;  
x.toUpperCase(); ← Numbers don't have this method
```

Performing invalid operations on data types

Using console.log()



COMPREHENSIVE EXERCISE

Practice Task

Build a Simple Interactive Page

Create a complete web page that combines **HTML structure**, **CSS styling**, and **JavaScript interactivity**. This exercise will reinforce everything you've learned today.

1 HTML Structure

- ✓ Create a heading with your name
- ✓ Add a paragraph describing yourself
- ✓ Include a button element
- ✓ Add an image (optional)

2 CSS Styling

- ✓ Change heading color
- ✓ Set paragraph font size
- ✓ Add background color
- ✓ Style the button

3 JavaScript

- ✓ Use `prompt()` to get user name
- ✓ Store name in a variable
- ✓ Show greeting with `alert()`
- ✓ Add button click handler



Challenge Yourself!

Try adding more features: different colors, more input fields, or additional interactive elements. Use the browser console to debug any issues!



DAY 2 COMPLETE

Wrap-up & Next Steps

Today's Key Concepts



CSS Fundamentals

Colors, fonts, spacing, and styling syntax



JavaScript Basics

Interactivity, alert(), and dynamic behavior



Variables

let, const, and storing data values



User Input

prompt() function and input handling



Debugging

DevTools, console, and error handling

Looking Ahead to Day 3



Tomorrow's Topics

- > Advanced CSS: Flexbox & Grid
- > JavaScript Functions
- > Event Listeners
- > DOM Manipulation



Project Preview

We'll start building a complete interactive web application combining all the skills learned so far!



Questions?

Don't hesitate to ask! Review the practice exercises and experiment with the code.